

Extraction of features from signals on cell-graphs to model biological image information

ÉCOLE POLYTECHNIQUE FÉDÉRALE DE LAUSANNE

Student:

Silvio Zanolì

Coordinators of the project:

Prof. Pascal Frossard
Dt. Bastien Padeloup
Dt. Mireille El Gheche



Suisse
2014

Presented the 8 June 2018

Contents

1	Introduction	3
1.1	The need for a cell-graph.....	3
1.2	Signal processing over a graph.....	3
1.3	The aim of this project.....	3
2	Data and Features extraction	4
2.1	Data exploration	4
2.2	Graph building.....	5
2.3	Features extraction	5
2.3.1	PD1 Features	5
2.3.2	Connectedness	6
2.3.3	Distances	6
2.3.4	Spectral	7
2.3.5	Signal.....	8
2.3.6	Borders	9
3	Correlation and Conclusions	10
3.1	Correlation.....	10
3.2	Conclusions	11
4	References	12

1 Introduction

The following report aims to summarize the work of the semester project started the 01/03/2018 and finished the 08/08/2018. In this introduction we will see the main idea and the motivation that led to the design and actuation of this project. The main paper from which the idea of a graph representation came is “Cell-Graphs: Image-Driven Modelling of Structure-Function Relationship”, Bülent Yener, 2017.

1.1 The need for a cell-graph

We can start, at first, defining what a cell-graph is: The main idea behind cell-graphs is to search and model a structural representation of the interactions that a biological system performs. More in detail, when we speak about cell-graphs we intend a graph representation of a slice of a tissue in which every vertex corresponds to a cell and every edge to some meaningful proprieties between cells, for example, the distance. In the past few years, the amount of data acquired by bio-medicine experts highly increased. Unfortunately, this growth in available data is not followed by a strong understanding of the overall interactions between biological systems. For examples: gene sequence and expression data illustrate the structure and activities of individual genes but does not explain how these genes collaborate to control cellular and tissue-scale functions. The previous example is on a lower level of abstraction than what we aim to do in this project that will be “cellule-centric” without going into the detail of DNA or inner proprieties. As stated before, the enormous amount of data available in the presents day permits us to create a structural representation that can capture similarities between structures with similar behaviours and this is the core idea of cell-graph: given the hypothesis that cells in a tissue organize themselves to perform specific functions, this technique aims to learn the structure-function relationship using graph theory. The full work-stack for cell-graph techniques is: image processing, feature extraction and selection and machine learning algorithms to establish a quantitative relationship between structure and function. This project, in particular, aims to perform the “feature extraction and selection” over graphs. This step has, as input, the image processing results and as output meaningful features that can be analysed later on with machine learning algorithms.

1.2 Signal processing over a graph

One of the main difference with the work of Bülent Yener consists in the use of a powerful mathematical framework: signal processing over graph domain. This method consists of the translation of classical signal processing tools like filtering and frequency analysis on the graph domain where the space upon which the signals are defined is not anymore R^n but the node space: this means that every node in the graph representation has one or more signals defined upon it. The main idea is to find a frequency representation of the said signal. We can do so by defining a Fourier base over the graph and then project the signal defined over the nodes upon it. With this tool is possible to compute some meaningful features of the cell-graph such as the smoothness of a signal or its frequency distribution. Without going into details, said W the weight matrix representing the graph, we can compute L the Laplacian matrix which is the equivalent of the Laplacian operator over graphs, once we obtained the Laplacian matrix we can observe that the eigenvalues of it are the eigenfrequencies of the Fourier base that we talked about (Ulrike von Luxburg, 2007). We will see later on what type of signals we can define.

1.3 The aim of this project

In this project we will apply the techniques discussed above to extract feature over a cell-graph built upon skin cells affected with cancer. The main idea is to find meaningful features that can describe the probability of survival or not (we will see the dataset in the following paragraph) to determine if a treatment is suitable or not for a patient: in the classical treatment of cancer, the therapy applied is often “broad range”, not specifically focused on the patient and this leads to a potentially harmful and useless use of medications. Nowadays, oncologists and researchers are trying to revert this paradigm. We believe that a cell-graph approach could be a step toward this goal.

2 Data and Features extraction

In this chapter we will discuss at first the dataset used and what the signals that we have represents. After that we will analyse what macro-type of features we thought to extract. We will then descend into details of the single features focusing, in particular, on the non-trivial ones. Finally, we will discuss briefly about the difficulties found during the projects

2.1 Data exploration

As we can see from Figure 2.1.1 the original dataset was composed of several photographs of slice of skin areas grouped accordingly to the patient from which those cells came from and coloured with a specific contrast agent from which was possible to extract information about the cells. We won't go any further into the original image extraction method: as stated in the introduction, this project takes as input the information extracted from the photographs and as output a series of features. The dataset used from this project is, hence, composed of:

- Positions: positions of each cellules on the photograph
- Type: type of each cellule which can be Tumorous, Stroma, Undefined
- Area: area of the cellules
- PD1 positivity: positivity to the PD1 protein which is an indicator of white blood cellules
- Keratin value: The amount of Keratin of each cell

From this dataset, in particular from position and type, is possible to construct 3 main graphs (Figure 2.1.2): A Full graph that takes into account every cellule discarding only the unclassified ones, a Stroma graph that considers only the Stroma cellules and a Tumour graph that look only at the tumorous cellules. Moreover, it's possible to associate to those graphs with a signal defined over their nodes. We can define three signals over the full graph, they are: Type, Area, keratin and PD1 positivity. Over the Tumorous and the Stroma graph we can define only the Area, keratin and PD1 positivity. For every patients we

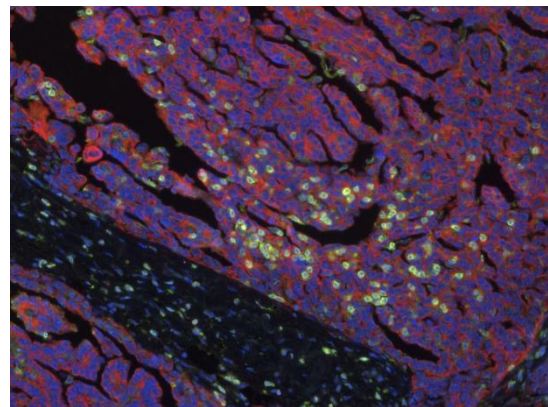


Figure 2.1.1 Example of a photograph of a skin tissue: we can see the red area composed mostly by cancerous cells while the darker is composed by stroma (healthy) cells

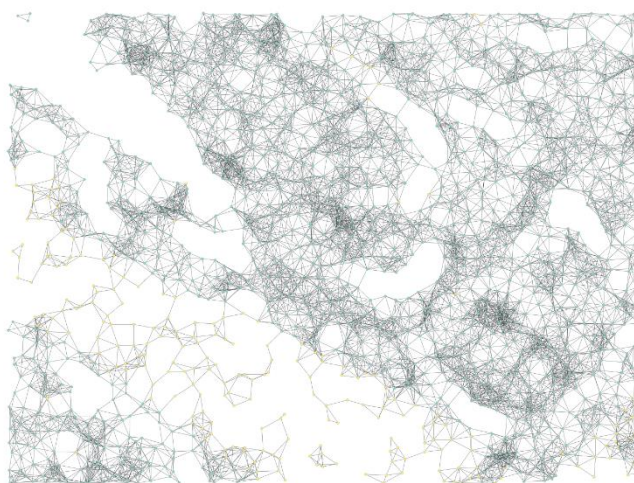


Figure 2.1.2 Full Graph representation of the Figure 2.1.1, The color of the nodes represents the type of the corresponding cell: yellow for the stroma and blue for the tumorous cells

have also their last known status: dead or alive. This last information permits us to distinguish the two cases when computing all the various features and visualize them separately. Unfortunately, this doesn't constitute at all a solid base for any classification: First we don't know the state of the tumour (advanced, beginning, etc..) and second, the only thing that we know for the "Alive" patient is that he/she wasn't dead at the moment of the sampling, this doesn't constitute a hint on any possible healing or worsening of the conditions so. This said, in the next paragraphs we will consider them as two different populations in order to be able to build a framework that can be possibly refined when more detailed data will be in possess.

2.2 Graph building

Once the coordinates of the cells were extracted, the l_2 distance was used as metric for the edges: the (simple) idea was to get the distance from a node to any other node and then keep only the more significant one, hence, the shortest few. This decision was preferred from the applying the K-NN (K nearest neighbours) because, in the case of isolated cells, this algorithm would lead to the definition of un-plausible and extremely long connections. We can see the results in the previous figure (Figure 2.1.2). In order to deal with the graphs, two main tools were used: Graph-tool for the managing of the signals and some features, and PyGSP in order to be able to use the signals and graph signal processing graph tools.

2.3 Features extraction

With “Feature” of a graph we define a value (or a series of them) that can summarize some characteristic of the graph for example, as we will see, the average number of nodes connected to one node. The main idea is to obtain information still understandable by human that at the same time can be used to train a machine-learning algorithm able at first to classify the status of the patient and in a second moment to estimate a treatment or the chance of survival to another. Once we defined all the graphs and set all the values of the signals over the corresponding nodes, we can start to elaborate them to extract some features. We extracted 58 features in total divided into 5 categories: PD1, Connectedness, Distances, Signal and Spectral. Hereafter we will list them with a brief (when possible) description of the most significant ones. This work was performed considering the work already performed by B. Yener, discarding some of the proposed features and including some others. A list of the extracted features can be seen in figure 2.3.1

2.3.1 PD1 Features

The PD1 protein is a protein present on the surface white blood cells and is used to detect if a cell is benign or malignant. Once detected we should be careful anyway: tumorous cells can, sometimes, mimic this protein, in this work we didn't take care of that eventuality, but we needed to point that out in order to show a possible important improvement that still need to be studied. The PD1 penetration features measure how much, in average, the PD1-positive cells penetrated inside a tumorous group of cells. The main idea that lies under this metric is the intuition that if a tumorous tissue has a high penetration of white blood cells, that could mean that the immune system of the patient is aware and fighting the tumour. Note that this is a speculation until we will have the results from the last step of the cell-graph process, the classification. In order to do so we apply a Breadth-first search (BFS) algorithm to all the PD1 positive cells and we check in how many steps we reach a group of stroma cells then we remove the all the ones that have a 0 distance: that mean that the cell under exam is on the border or inside a group of stroma cells hence is not penetrated, its count would lower the average of a feature that aims to summarize the penetration of PD1-positive cells, not their count.

```
{'full': {'PD1': {'PD1penetration': 0},
'connectedness': {'NoComponents': 3,
'avgDegree': 338.7643901621581,
'clusterCoeffAvg': 0.5585075331023004,
'clusterCoeffGlobal': 0.5599876112541975,
'endpoint': 0.00423728813559322,
'giantComponentRatio': 0.9983050847457627,
'isolatedNode': 0.001694915254237288},
'distances': {'avgDiameter': 1382.741822772245,
'avgDiameter90': 1253.5808523003654,
'avgEccentricity': 1050.9436368658487,
'avgEccentricity90': 1023.3036752696496,
'avgNumCentralPoint': 0.9983050847457627,
'avgNumEdge': 5865,
'avgNumVerts': 1178,
'avgPercCentralPoint': 0.000846021258259121,
'avgRadius': 705.4483408447911,
'avgRadius90': 705.4483408447911,
'avgcloseness': 514.008819081917,
'cutLenStroma': 323,
'cutLenTum': 251,
'numEdges': 5865,
'numVert': 1180},
'signal': {'PD1Energy': 27.000000000000004,
'PD1FirstFreq': 0.5777126162775186,
'PD1FirstFreqFilt': 6.48107409000425e-17,
'PD1SecondFreq': 0.9522594865349502,
'PD1SecondFreqFilt': 1.5782178061186604e-17,
'PD1ThirdFreq': 0.6965587032604994,
'PD1ThirdFreqFilt': 2.4242080189253758e-11,
'areaEnergy': 205318686.00000054,
'areaFirstFreq': 13264.94893935381,
'areaFirstFreqFilt': 230.83414157911525,
'areaSecondFreq': 1036.9676358116017,
'areaSecondFreqFilt': 266.89260851608947,
'areaThirdFreq': 226.9166366820382,
'areaThirdFreqFilt': 171.75882106468097,
'categoryEnergy': 842.0000000000001,
'categoryFirstFreq': 22.992049167703595,
'categoryFirstFreqFilt': 0.9914708355691966,
'categorySecondFreq': 0.5399391900228139,
'categorySecondFreqFilt': 0.9829066620425435,
'categoryThirdFreq': 2.4064251612683227,
'categoryThirdFreqFilt': 0.995112963784515,
'keratinEnergy': 14616.558477999988,
'keratinFirstFreq': 70.19409590887788,
'keratinFirstFreqFilt': 0.4546237442271439,
'keratinSecondFreq': 8.059226786005535,
'keratinSecondFreqFilt': 0.31700313249871104,
'keratinThirdFreq': 41.16871445141019,
'keratinThirdFreqFilt': 0.32911093622726045},
'spectral': {'eigen0L': 1,
'eigen1L': 2,
'eigen2L': 0,
'energyL': 1392400.0,
'largestEignW': 513.8015824101507,
'secondLargestEignW': 505.58147672776045,
'traceL': 1180.0}}}
```

Figure 2.3.1 Here we can observe the partitioning of the 58 extracted features (un-normalized) into their categories: note that the PD1 category only hold one features, that is because the PD1 penetration is not similar to any other feature as we discuss later

2.3.2 Connectedness

This family of features aims to capture the connection behaviour of the graph. Those are important features in classical graph theory plus they remain very interpretable even at an intuitive level.

The number of components is the number of disconnected subgraphs in the present graph, this feature allows us to see how much the graph under analysis is fractured. This may seem useless if we simply compute it on a “Full” graph but if we compute it for the “Stroma” graph and for the “Tumour” graph independently, this may show the sparseness of one with respect to the other.

The average degree is the average number of edges for each node: this is the sum of every edge multiplied by its weight times 2, divided by the number of nodes. The multiplicative factor, 2, is because we are dealing with undirected graphs hence every edge should be counted for both node to which the edge is attached to. Given the technique we used to build the graph, a high average degree also holds some information about distances (high average degree mean that the nodes have a high number of edges that were built and thresholded based on the Euclidian distance). A high average can code the fact that a vast number of cells lay in a small area.

The average cluster coefficient is the sum of the cluster coefficients of every node divided by the number of nodes. The global clustering coefficient is indeed the number of triangular structure present in the graph divided by the total number of triplets of the graphs, multiplied by 3. The multiplicative factor of 3 is given by the fact that the actual definition of global clustering coefficient is: the number of closed triplets divided by the number of all triplets, but the number of closed triplets is equal to three times the number of triangles.

The endpoint is the average number of nodes with only one edge connected to him. The isolated node is indeed the number of nodes with no edge connected, given the technique we used to build the graph, this show how many nodes are too far from the closer node given the threshold.

The giant component ratio is the number of node in the biggest disconnected subgraph averaged with respect to the total number of nodes present. We can repeat the same reasoning we had for “number of components” to justify this feature.

2.3.3 Distances

This family of features purpose is to have some features responsible for the showing of metric-related features. As before, those are typical features in classical graph theory and they remain interpretable even at an intuitive level. Note that every type of distance (and features derived from them) is defined as infinity if measured between two disconnected components. Because we have a graph composed of several disconnected components that would lead to only infinity values for all those features if we don't take care of this. To obviate to this problem, we computed those features for all the disconnected components bigger than a threshold (computed dynamically with respect to the biggest component present in the graph) and then summed them up with a different weighting for all of them. This weight is given by the ratio between the node in the component under exam and the total number of nodes: it would make no sense to apply an average because then small component with an abnormal value would radically change the value of the feature. One other possible solution to this problem may have been to compute the following features (that we will see are based on the average of some metric computed for each node) and simply discard the infinity values. This is a valid solution even if computational inefficient.

The average diameter is the average-weighted, longest path among all the possible shortest path of the graph, the shortest path among the two most distant nodes, we can observe that this is also re-definable as the longest path between the eccentricity of every node. The 90% diameter is based on the same idea but computed on the 90% of the node ordered with respect to their degree in order to exclude possible outliers. We decided to keep both of them: their difference can represent the number of outliers in this metric, that could potentially be a useful information. The average radius is really similar to the diameter metric: this is the shortest path among the eccentricity of every node.

The average eccentricity is the average between all the longest path among all the possible shortest path from a node to another. In other words, given a node, we computed all the shortest path to any other nodes in the present component and then we kept only the longest among them, we repeated this for every node in the component and then averaged with respect to the number of nodes. For the average eccentricity at 90% we can repeat the same reasoning we made for the 90% diameter. We can interpret the eccentricity as the shortest distance between the selected node and the farther node from it, a high value of it can be seen as a consequence of the node being positioned on a peripheral part of the graph. The average eccentricity can be hence interpreted as the measure of how much the graph is composed of peripherals or central nodes.

The average number of central nodes is the number of nodes that have an eccentricity equal to the radius. As the name suggest, those nodes can be seen as the centre of the graph. The percentage number of central nodes is the percentage of nodes that are central points in the current component. These two features are strictly correlated with the average eccentricity without being redundant: a high or low value of average eccentricity doesn't imply a high/low average number of central nodes.

The closeness of a node is the inverse of the average shortest distance between said node and any other node, as the name may suggest, this metric describe "how close" a node is to any other in the graph. Here we simply computed the inverse of the closeness of every nodes and then averaged it with respect to the number of nodes. A high average inverse closeness could be representative of a really sparse graph.

The "cut length" we computed (with respect to tumours and to stroma) deserve a more extensive explanation and thus we will talk about them in a different and dedicated paragraph at the end of this section.

2.3.4 Spectral

This family of features, already defined in the paper of Bülent Yener, aims to find some particular value related to the various (two) matrix representation of the graph. Note that those features are not understandable at an intuitive level as the previous ones but remain of big relevance for classical graph theory. Those features were computed with the help of the PyGSP library. This library was thought with the main purpose to perform graph signal processing (hence spectral processing). A small modification to this librarie was needed in order to be able to use it: we will see hereafter that, in order to extract some features, a full diagonalization of the normalized Laplacian was needed, the list of the eigenvalues obtained with this diagonalization need to be bounded between 0 and 2 included, we won't go any further into the details of the motivation of this (Ulrike von Luxburg, 2007). In the PyGSP library this diagonalization assert that the largest diagonal value was exactly less-equal than 2. This would make sense if we don't consider the fact that, if the largest eigenvalues reach the maximum value of 2, because this value is computed with a recursive algorithm with double precision float, the result could be slightly bigger than 2. We simply modified the library introducing a tolerance around the larges eigenvalue. This collection of Features is called "Spectral" because it's possible to read the list of the eigenvalues of the Laplacian matrix as the eigenfrequencies defined over the graph.

The number of eigenvalues equal to 0 (algebraic multiplicity of $\lambda = 0$) ie equal to the number of disconnected components minus the number of isolated nodes. Because the eigenvalues of the Laplacian matrix of a disconnected graph are the union of the eigenvalues of the singles disconnected components, is possible to show that the eigenvalue of the Laplacian matrix of a single disconnected node is not defined.

The number of eigenvalues equal to 1 may have some importance: in some particular graphs, this eigenvalue is interpretable as the axis of symmetry for the spectrum of the eigenfrequencies.

The number of eigenvalues equal to 2 is the number of bipartite connected components present in the graph. Bipartite components are defined as a graph composed of 2 group of nodes nodes where every node in one group is connected to one or more node in the second one without being connected to any node in the same group. As said, we won't go into the mathematical demonstration of this statement that can be found in the citation (Ulrike von Luxburg, 2007).

2.3.5 Signal

As said before, one of the main difference with the features extracted from Bülent Yener is the computation of features defined based on signals defined over the nodes. As described in the theory of graph signal processing, once we have a graph we can compute the Laplacian matrix of it and then extract the eigenvalues of it. Those eigenvalues can be read as the eigenfrequencies defined on the graph. Once we have the eigenfrequencies it's possible to project the signal defined on the nodes on the frequency space. We remember that we have 4 main signals: PD1

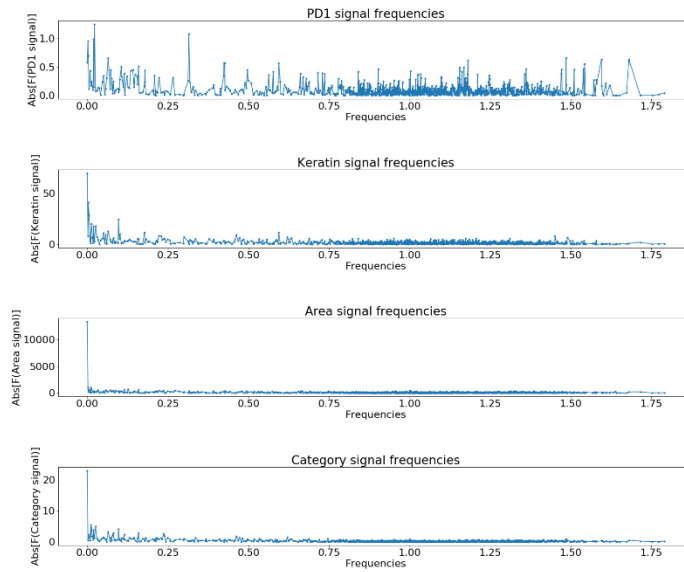


Figure 2.3.2 The projection of the signals over the eigenfrequencies space for a "Full" graph

Category. Hereafter we will see the main features extracted from the signals. The projection of the signals over the space frequencies for one graph can be found in Figure 2.3.3

The first, second and third frequencies representation of the PD1 signals are the first three taps, corresponding to the first three eigenfrequencies, of the PD1 signals. From Figure 2.3.2 we can see that the Frequencies representation of this signal has high values also in the high frequencies. This is because the PD1 is a signal that identifies the presence/absence of the white-blood cells and hence that can be sparse hence with really high frequencies (we could think about it as some Dirac pulse positioned on some precise nodes). In this case we took only the first three eigenfrequencies just to show the potentiality of this framework, nothing avoids us to just increase how many eigenfrequencies we keep.

The first, second and third filtered frequencies representation of the PD1 signals are the first three taps, corresponding to the first three eigenfrequencies, of the PD1 signals once it gets filtered with a lowpass filter (in this case a heat diffusion kernel). Again, the choice of the parameter of the lowpass filter was chosen just to show the potentiality of this framework. We won't talk about the filtered version of all the other signals signals, the reasoning is the same as we have seen here.

For first, second and third frequencies of the Keratin we can repeat the same exact reasoning we had for the PD1 frequencies: Here we can see that his value decrease faster than the PD1, this mean that this is a signal that tends to have a slow variation over the graph. The physical meaning of this need to be discussed with the doctors in order to have a better understanding of this features

In this case, we can see that the frequency representation of the signal defined by the area is almost not varying at all: the first eigenfrequencies can be interpreted, in the classical Fourier domain, as the continuous offset applied to the signal, here we can apply the same idea and see that the area tends to remain constant along all the graphs.

We also tried to use the category as a signal to analyse. Here we can see that the result is what we were expecting: in this graph we had really big clusters for both stroma cells and tumorous cells. Inside those clusters the signal remains constant, changing only on the border. This leads the transformed signal to be low valued but with some ripple given by the fact of the presence of a sharp border. This can be really representative of the more or less fragmentation of the tumorous clusters (or of the stroma clusters).

2.3.6 Borders

The concept that lay behind the border features is really simple as one can think: the nodes that lay on the border between two types of cells (stroma and tumour). The feature extracted is the number of nodes that lay on the border (for both stroma and tumour). At first, we implemented it by simply analysing the edges of every node and checking if among the borders of it there was a node of the opposite type. Doing like this it raises several problems: first we don't have any information on the border connecting them and second it may ignore some node that still is part of the border. This case happens when we have a border

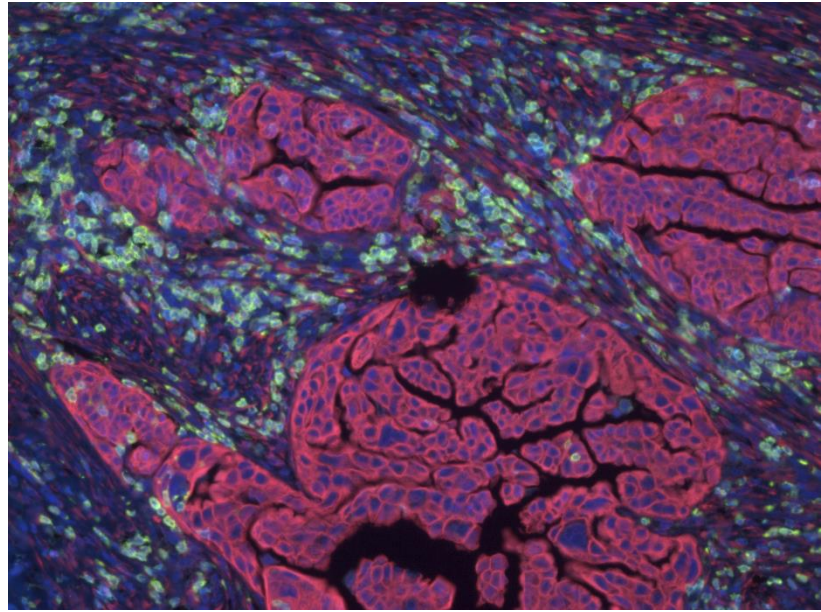


Figure 2.3.3 photograph of a skin tissue: we can see the red area composed mostly by cancerous cells while the darker is composed by stroma (healthy) cells

in which, at a certain point, the two types of cells get more distant than the threshold used to create the graph, in this case, the node that lay in this separated section on the border doesn't have any connection to any node of the different type. This problem was solved by implementing an advanced version of what said before (in the explanation we will consider the border of one type of cell with respect to the second, the algorithm here described is fully symmetric): at first, we search a cell that lay on the border with the same techniques as before then we define this as seed and we search a chain of node on the border that has inside the seed. We add all the border found to a list of visited nodes in order to not go through them multiple times and then we repeat the algorithm for all the possible seed that we can find. Finished this step we have a list of chains that lay on the border of the graph, at this point for each of

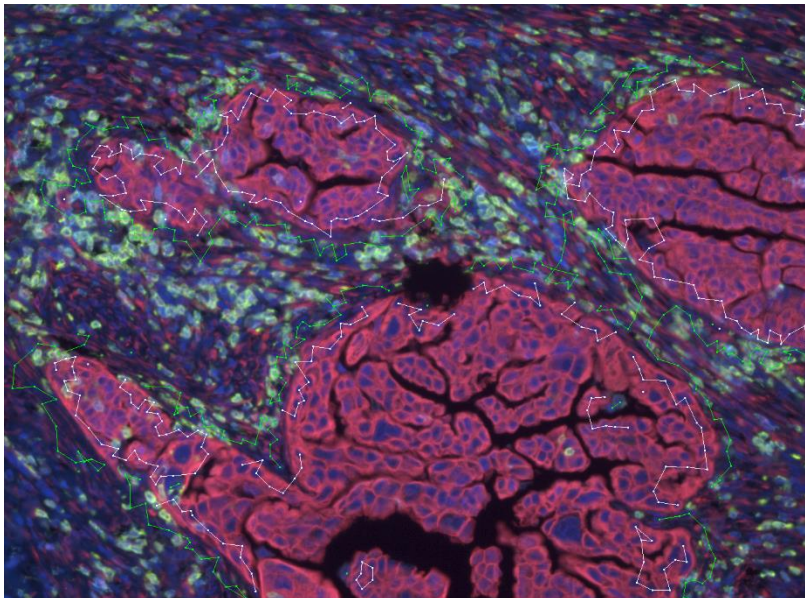


Figure 2.3.4 The border computed with the previous algorithm: in green the border of the stroma with respect to the tumor and in white the opposite

them we compute the minimum path from both the ends of the chain under exam to both ends of every other chains and then we keep the shortest one, adding the nodes and the edge to the list of the borders. To avoid every chunk of border to be connected to every other chunk (creating only one big border) we define a threshold, if the connecting chunk between two borders is longer than this threshold we simply discard it. An example of this algorithm can be found in the following Figures 2.3.3 and 2.3.4

3 Correlation and Conclusions

We want to finish this report showing how the seen features correlate one to the other once normalized and concluding with a brief comment on the work performed

3.1 Correlation

To understand how our features correlate one to the other we computed them for a number of patients (with several tissue samples for each patient) and then we divided in 2 groups: features computed on tissues of dead patients and on tissue of alive patients, each of one divided in 3 group, one for each of the 3 possible graph: Full, Stroma and Tumor (the last two simply obtained by taking only the stroma or only the Tumorous cells from the Full graph). Here we will see the results only for the Full features because is the one that holds more information. We should not ignore, anyway, the remaining 2: as said before, some features could be significant just if we compare them for those 2 groups. Once we computed the features and divided them into those 6 groups we normalize them based on the values of the same features in the same group. In Figure 3.1.1 is possible to see the correlation matrix computed

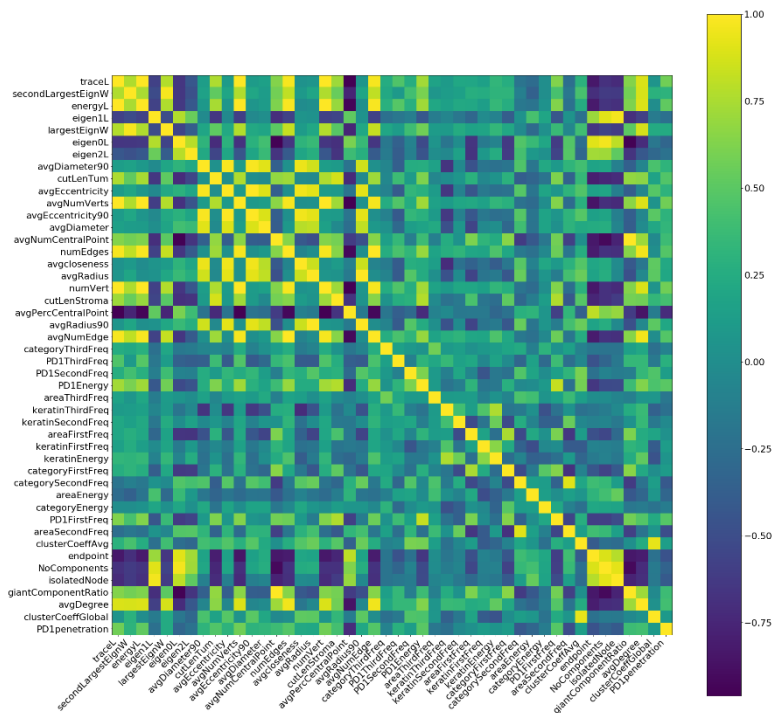


Figure 3.1.1 Correlation Matrix for the features extracted from the Full graph of alive patients

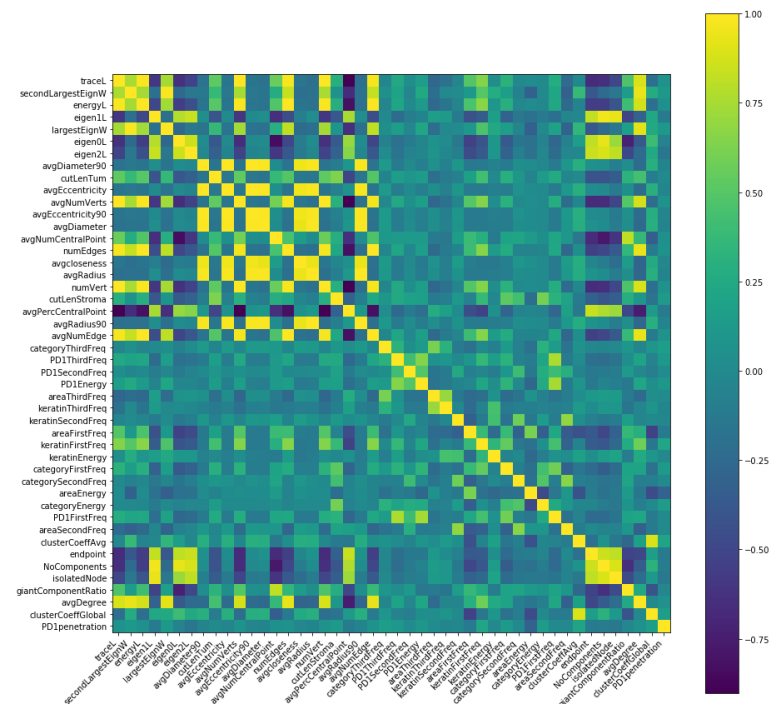


Figure 3.1.2 Correlation Matrix for the features extracted from the Full graph of alive patients

for the features extracted from the full graph of alive patient while in figure 3.1.2 we can see the same thing but for dead patients. As we were expecting, all the “classical” features highly correlate (negatively or positively) among them while the PD1 penetration and all the frequencies related features seems to don’t correlate with the others. If this would be true, then it would mean that we found a set of features highly independent one from the other hence representative of different aspects of the graph. In reality this assumption may be bold: the dataset analysed is too small (18 patients among dead and alive, 6 tissues in average for all of them) to have a right approximation of the variance and the mean of each features plus we always must to remember the mantra of data science: “Correlation does not mean causality”. This result can, anyway,

makes us hope for the best in the future developments and in the application of a machine learning algorithm to the extracted features.

3.2 Conclusions

Starting with the graph representation of the cells of a tissue and 4 signals defined over those graphs, we were able to extract a total of 58 features. Every graph can, with this algorithm, potentially be summarized by a 58-dimension vector that can be elaborated from a machine learning algorithm in order to perform various tasks (healing estimation, survival probability, etc.). We have seen that it seems that the “new” (i.e. not already implemented by Bülent Yener) features extracted seems to be independent from the others. Unfortunately, we hadn’t the possibility of testing if such extracted features are “good” or “bad” for any of said tasks mainly because we are still missing the final part of the cell-graph process which is, indeed, the design of a machine learning algorithm. Future development will be aimed to extract even more features from a bigger dataset, estimate the robustness of this algorithm to the disturbance and to design an algorithm capable to process those features.

4 References

- Bülent Yener, " Cell-graphs: image-driven modeling of structure-function relationship", Rensselaer Polytechnic Institute, Troy, NY Communications of the ACM, Volume 60 Issue 1, January 2017
- Ulrike von Luxburg, "A Tutorial on Spectral Clustering", Max Planck Institute for Biological Cybernetics Spemannstr. 38, 72076 Tübingen, Germany, Statistics and Computing, 17 (4), 2007